SLR: An Automated Synthesis Framework for Scalable Logical Reasoning

Lukas Helff[1,2], Ahmad Omar[1], Felix Friedrich[1,2], Wolfgang Stammer[1,2], Antonia Wüst[1], Tim Woydt[1], Rupert Mitchell[1,2], Patrick Schramowski[1,2,3,4], Kristian Kersting[1,2,3,5]
[1]TU Darmstadt [2]hessian.AI [3]DFKI [4]CERTAIN, Germany
[5]Centre for Cognitive Science, Darmstadt

Code&Data: https://github.com/ml-research/ScalableLogicalReasoning

arXiv:2506.15787v2 [cs.AI] 23 Jun 2025

Abstract

We introduce SLR, an end-to-end framework for systematic evaluation and training of Large Language Models (LLMs) via **S**calable **L**ogical **R**easoning. Given a user's task specification, SLR enables scalable, automated synthesis of inductive reasoning tasks with precisely controlled difficulty. For each task, SLR synthesizes (i) a latent ground-truth rule, (ii) an executable validation program used by a symbolic judge to deterministically verify model outputs, and (iii) an instruction prompt for the reasoning task. Using SLR, we create SLR-BENCH, a benchmark comprising over 19k prompts spanning 20 curriculum levels that progressively increase in relational, arithmetic, and recursive complexity. Large-scale evaluation reveals that contemporary LLMs readily produce syntactically valid rules, yet often fail at correct logical inference. Recent reasoning LLMs do somewhat better, but incur substantial increases in test-time compute, sometimes exceeding 15k completion tokens. Finally, logic-tuning via SLR doubles Llama-3-8B accuracy on SLR-BENCH, achieving parity with Gemini-Flash-Thinking at a fraction of computational cost. SLR is fully automated, requires no human annotation, ensures dataset novelty, and offers a scalable environment for probing and advancing LLMs' reasoning capabilities.

1 Introduction

Logical reasoning is a fundamental aspect of intelligence, yet state-of-the-art AI systems still lag in tasks requiring robust reasoning and systematic generalization (Delfosse et al., 2025; Kostikova et al., 2025; Woydt et al., 2025; Wüst et al., 2025; Helff et al., 2025; Sinha et al., 2019). Current benchmarks largely focus on *deductive* reasoning, where conclusions are necessarily entailed by premises. This includes math problems (Hendrycks et al., 2021b) and logic puzzles (Lin et al., 2025; Xie et al., 2025; Liu et al., 2025). In *inductive* reasoning, general rules or patterns are inferred from specific examples or observations, which has been shown to be especially challenging and underexplored for LLMs (Luo et al., 2024; Xie et al., 2024) (also see Tab. 1).

Current evaluation frameworks commonly employ constrained formats (e.g., multiple-choice) or rely on other LLMs to serve as judges (Patel et al., 2024a; Lin, 2024; Lin et al., 2024), making it difficult to assess whether models genuinely understand logical structure or are merely exploiting superficial patterns in the data. Moreover, as training sets grow, benchmark items or their paraphrases increasingly overlap with pre-training data, making apparent reasoning abilities potentially just memorization (Shojaee et al., 2025; Xie et al., 2024).

This paper introduces **SLR** (Scalable Logical Reasoning), an open-source framework for evaluating and training models in logical reasoning. From a user-defined logic task (Fig. 1, left), the task synthesiser (center) automatically generates novel inductive logic programming (ILP) tasks of controllable complexity. Each task comes with (i) a latent ground-truth rule, (ii) an executable validation program, and (iii) an instruction prompt for the reasoning task. The ground-truth rule serves as the reference answer, while the validation program deterministically evaluates any candidate hypothesis. SLR supports both systematic model evaluation (Fig. 1, top right) and downstream model training, either via supervised finetuning or reinforcement learning with rewards provided by the symbolic judge (Fig. 1, bottom right). Because both generation and validation are symbolic and automated, SLR requires no human annotation and avoids overlap with existing corpora.

Leveraging SLR, we present a first instantiation, **SLR-BENCH** (Fig. 2), a 19k task benchmark that forms a twenty-level curriculum of increasing logical complexity. These levels are further organized into four curriculum tiers: *basic*, *easy*, *medium*, and *hard*. Each task is unique, with a

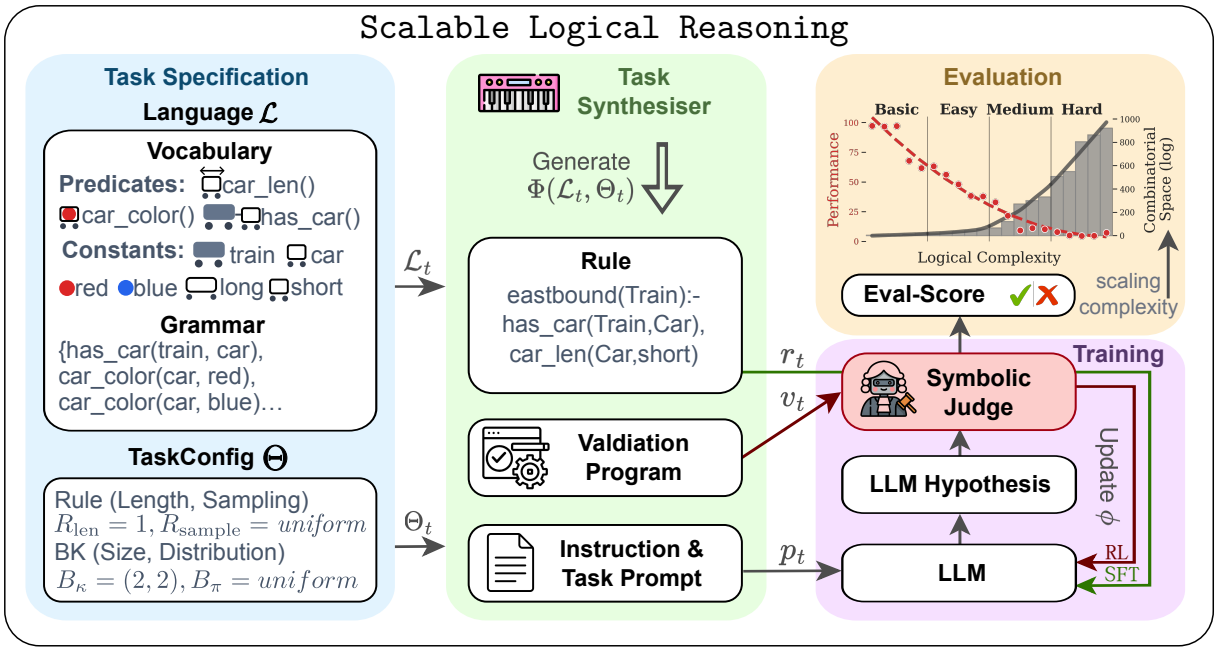


Figure 1: Overview of the **SLR** Framework, including formal task specification, automated task synthesis, training, and evaluation. **Left (blue):** Language defines vocabulary and grammar for automated task generation; Task Config specifies hyperparameters for background and rule generation. **Middle (green):** The ILP Task Synthesiser automatically generates ground-truth rules, validation programs, and instruction prompts according to the defined language and config. **Right (purple):** Training LLMs on logic tasks, with updates provided via either cross-entropy loss for SFT or symbolic judge feedback for RL. **Right (orange):** Evaluating LLMs on synthesized tasks, using the symbolic judge to provide automated scoring of LLM hypotheses. Arrows denote data and control flow through task synthesis, prompting, LLM evaluation, and downstream training loops.

precise and systematic assessment of logical reasoning skills. Running a large-scale evaluation, we find that LLMs almost always produce syntactically valid rules, yet struggle with complex reasoning. Most models readily solve the initial levels, indicating that contemporary LLMs possess the very basic capabilities for logical inference. However, as we move towards more logically complex tasks, we observe a steady decline in model performance. Beyond evaluation, SLR enables logic-tuning, allowing Llama-3-8B trained on SLR-BENCH to outperform much larger reasoning models while generating significantly fewer tokens.

In sum, our contributions are: (i) SLR, an open framework for automated synthesis and symbolic evaluation of logical reasoning in LLMs. (ii) SLR-BENCH, a 19k task benchmark that forms a 20-level curriculum of increasing logical complexity, enabling training and evaluating for logical reasoning across a controlled complexity spectrum. (iii) We evaluate LLMs on SLR-BENCH, highlighting key insights and trade-offs. (iv) We logic-tune LLMs with SLR, boosting reasoning ability.

2 Related Work

Evaluating LLMs' Logical Reasoning. Tab. 1 summarizes several logical reasoning benchmarks and frameworks, contrasting their goals, design choices, and trade-offs. LogiQA/2.0 (Liu et al., 2020, 2023) and FOLIO (Han et al., 2022) provide human-authored tasks testing primarily deductive reasoning (via multiple-choice or proof formats). AbductionRules (Young et al., 2022) focuses on abduction; models must generate plausible missing facts (explanations) that account for the observation. bAbI (Weston et al., 2015) and CLUTRR (Sinha et al., 2019) are synthetic datasets that consist of diverse reasoning tasks, while bAbI focuses on toy QA tasks in simulated environments, CLUTRR targets inductive relational reasoning through short kinship stories designed to test systematic generalization. BIG-Bench (Kazemi et al., 2025; Suzgun et al., 2023), HLE (Phan et al., 2025), LogiGLUE (Luo et al., 2024), FineLogic (Zhou et al., 2025) aggregate existing datasets to provide testbeds across multiple task types. Proofwriter (Oyvind et al., 2021) and PrOntoQA (Saparov and He, 2023) sample entail-

Table 1: **Comparison of logic reasoning benchmarks and frameworks. Reasoning Type**: Principal logical inference type(s) (deduction, induction, abduction, or mix). **Creation**: How the dataset is created (synthetic, human-annotation, DS collection). **Evaluation**: Output scoring (symbolic, MCQA/Auto, LLM, exact match (EM)). **Task Synthesis**: Can the framework generate new tasks? **Custom Tasks**: Can users create custom tasks (e.g., define a new language, grammar, or task setup)? **Curriculum Learning**: Is a staged, fine-grained progression of difficulty/curriculum available? **Scalable Difficulty**: Can the complexity of generated tasks be scaled arbitrarily? (✓: fully supported, ✗: not supported, (✓): partially/limited)

Dataset	Reasoning Type	Data Creation	Evaluation Methodology	Task Synthesis	Custom Tasks	Curriculum Learning	Scalable Complexity
LogiQA (Liu et al., 2020)	Deduction	Human	MCQA/Auto	✗	✗	✗	✗
LogiQA 2.0 (Liu et al., 2023)	Mix	Human	MCQA/Auto	✗	✗	✗	✗
FOLIO (Han et al., 2022)	Deduction	Human	EM	✗	✗	✗	✗
AbductionRules (Young et al., 2022)	Abduction	Synthetic/Human	EM	✗	✗	✗	✗
FineLogic (Zhou et al., 2025)	Deduction	DS collection	Symbolic/LLM	✗	✗	✗	✗
HLE (Phan et al., 2025)	Mix	DS collection	LLM	✗	✗	✗	✗
Big-Bench (Phan et al., 2025)	Mix	DS collection	Mix	✗	✗	✗	✗
LogiGLUE (Luo et al., 2024)	Mix	DS collection	MCQA/Auto	✗	✗	✗	✗
Multi-LogiEval (Patel et al., 2024b)	Deduction	Synthetic	Symbolic	✗	✗	✗	✗
CLUTRR (Sinha et al., 2019)	Induction	Synthetic	EM	✗	✗	✗	✗
KOR-Bench (Ma et al., 2024)	Mix	Synthetic/Human	EM/Symbolic	(✓)	✗	✗	✗
PrOntoQA (Saparov and He, 2023)	Deduction	Synthetic	EM	✓	✗	✗	✗
bAbI (Weston et al., 2015)	Mix	Synthetic	EM	✓	✗	✗	✗
SynLogic (Liu et al., 2025)	Deduction	Synthetic	EM	✓	✗	✗	✗
FLD (Morishita et al., 2024)	Deduction	Synthetic	Symbolic	✓	✓	✗	(✓)
K&K (Xie et al., 2025, 2024)	Deduction	Synthetic	EM	✓	✗	(✓)	(✓)
ZebraLogic (Lin et al., 2025)	Deduction	Synthetic	EM	✓	✗	(✓)	(✓)
SLR (ours)	Induction	Synthetic	Symbolic	✓	✓	✓	✓

ment questions from ontologies. KOR-Bench (Ma et al., 2024) blends human-constructed rules with synthetic QA generation. Formal Logic Deduction (FLD) (Morishita et al., 2024) provides a formal framework for generating synthetic deductive reasoning tasks. Multi-LogiEval(Patel et al., 2024b) provides a benchmark for multi-step deductive reasoning. SynLogic (Liu et al., 2025) generates logic problems from 35 fixed deduction tasks, while ZebraLogic (Lin et al., 2025), and K&K sandbox (Xie et al., 2025, 2024; Mondorf and Plank, 2024) synthesize arbitrarily many grid-world deduction puzzles with a coarse, manually tuned curriculum but still rely on a fixed task repertoire and are evaluated via exact matching. Altogether, these benchmarks lack key features for scalable logical reasoning, such as extensibility, systematic curriculum control, and task synthesis with custom language and task setups, all of which are provided by our framework (see Tab. 1).

Limits and Promises of Reasoning LLMs. LLMs like GPT-4 (OpenAI et al., 2024), Llama-3 (Meta et al., 2024), and Qwen (Bai et al., 2023) can handle basic reasoning and coding tasks but often struggle with true abstraction (Shojaee et al., 2025; Xie et al., 2024). Recent *reasoning LLMs* attempt to bridge this gap by scaling *test-time* compute. Systems like OpenAI's o1/o3 (OpenAI, 2025) or DeepSeek-R1 (DeepSeek-AI et al., 2025) generate and re-rank thousands of reasoning traces per query, achieving state-of-the-art results on, e.g., math or coding (Quan et al., 2025; Hendrycks et al., 2021a,b; Rein et al., 2024; Gao et al., 2024; Fourrier et al., 2024; Clark et al., 2018). However, these gains come at a steep cost (Fan et al., 2025; Kim et al., 2025). Some studies question whether such models truly learn logical structure or merely exploit surface-level patterns (Fan et al., 2025; Shojaee et al., 2025; Xie et al., 2024). Curriculum learning has been shown to enhance training robustness and generalization (Bengio et al., 2009; Bursztyn et al., 2022), yet no prior framework offers a flexible framework for task synthesis for automatic curriculum generation with symbolic evaluation for reasoning at scale. SLR addresses this gap.

3 SLR: Automatic Logic Benchmark Synthesis

This section introduces SLR, a methodology for systematically generating, evaluating, and training on Inductive Logic Programming (ILP) tasks at scale. The core motivation for SLR is to provide a principled, automated, and scalable means of constructing diverse and challenging logical reasoning

tasks embedded within natural language prompts. The SLR pipeline follows the three-stage process of Task Specification, Task Synthesis, and Evaluation & Training, as illustrated in Fig. 1 and detailed in the following sections.

Task Notation. Our methodology adopts a structured design for ILP tasks, following the Learning-from-Entailment (LFE) paradigm (Raedt, 1997) and using the Prolog language for task representation. Each ILP task $\mathcal{I}$ is defined as follows:

$$\mathcal{I} = \left(B, E^+, E^-\right)$$

where B is the set of background ground atoms (i.e., context or facts for the task), E^+ (E^-) is the set of positive and negative labeled examples (i.e., ground atoms for which the target predicate should or should not hold. A candidate hypothesis H *solves* the task if and only if:

$$B \cup H \models E^+ \quad \text{and} \quad B \cup H \not\models E^-.$$

Here, $\models$ denotes logical entailment, as verified through symbolic execution in Prolog under the closed-world assumption.

3.1 Task Specification (Input)

The SLR synthesizer is controlled by a set of user-defined inputs that specify the structure and content of the generated tasks. Specifically, these inputs consist of the Task Language $\mathcal{L}$, which defines the logical vocabulary and grammar, and the Task Configuration Θ, which encodes parameters to control the generation process (see Fig. 1, left).

Language Specification ($\mathcal{L}$): We define a language $\mathcal{L} = (\mathcal{V}, \mathcal{G})$ that specifies the building blocks for task generation. This involves formulating a vocabulary $\mathcal{V}$ of allowed syntactic elements and a grammar $\mathcal{G}$ that imposes semantic constraints.

(i) Vocabulary $\mathcal{V}$: A set of constant, function, and predicate symbols that form the syntax for generating rules, examples, and background knowledge. The vocabulary induces the Herbrand base $\mathrm{HB}(\mathcal{V})$, which is the set of all syntactically valid ground atoms (facts).

(ii) Grammar $\mathcal{G}$: A set of semantic rules that filter the Herbrand base to include only meaningful atoms, $\mathrm{HB}_{\mathcal{G}}(\mathcal{V})$. In this way, the grammar introduces a bias so that only semantically coherent facts are produced. For instance, the grammar ensures that a color is assigned to a car (car_color(car, red)) but not to semantically incompatible objects.

Task Configuration (Θ): The configuration parameters give fine-grained control over the generation process via the tuple:

$$\Theta = \langle R_{\text{sample}}, R_{\text{len}}, B_\pi, \kappa \rangle$$

(i) Rule Sampling Policy (R_{sample}): The policy for synthesizing the ground truth rule $R^\star$. Rules can either be sampled randomly (Uniform Sampling) or generated via an LLM (LLM-Guided Generation). For LLM-Guided Generation, a model is tasked to generate rules that contain arithmetic, recursions, variables, cuts, or comparison operators.

(ii) Rule Length (R_{len}): An integer specifying the number of literals in the body of the ground-truth rule $R^\star$.

(iii) Background Sampling Policy (B_π): A probability mass function that assigns a selection probability to each ground atom in the grammar-filtered Herbrand base $\mathrm{HB}_{\mathcal{G}}(\mathcal{V})$, enabling designers to encode priors on the data distribution (e.g., *uniform*). We also include *mirror* sampling, where backgrounds for (E^+, E^-) are identical except for ground atoms relevant to $R^\star$.

(iv) Problem Size ($\kappa = (\kappa_{\text{pos}}, \kappa_{\text{neg}})$): A tuple specifying the target number of positive ($\kappa_{\text{pos}} = |E^+|$) and negative ($\kappa_{\text{neg}} = |E^-|$) examples. This directly controls the size and class balance of each generated ILP task.

3.2 ILP Task Synthesiser (Generation)

The core of SLR is the ILP Task Synthesizer (Fig. 1, center), an automated process detailed in Algorithm 1. Given a high-level task specification, the synthesizer generates complete and solvable ILP problems. The process consists of two main phases: rule synthesis and background synthesis.

1. Rule Synthesis (Alg. 1, line 2). The process begins with the RULEGENERATOR creating a latent, ground-truth rule $R^\star$. This rule represents the underlying logical pattern that a model is expected to induce. The generation is guided by pre-defined parameters (R_{len}, R_{sample}) that control the length and generation policy for the rule. The resulting

Algorithm 1 Task Synthesiser

Require: $\mathcal{L}$, B_π, κ_{pos}, κ_{neg}, $R_{\text{sample}}, R_{\text{len}}$
1: $B \leftarrow \varnothing, E^+ \leftarrow \varnothing, E^- \leftarrow \varnothing$
Rule Synthesis
2: $R^\star \leftarrow \text{RULEGENERATOR}(\mathcal{L}, R_{len}, R_{sample})$
3: **while** $|E^+| < \kappa_{\text{pos}}$ **or** $|E^-| < \kappa_{\text{neg}}$ **do**
Background Synthesis
4: $b \leftarrow \text{BACKGROUNDGENERATOR}(\mathcal{L}, B_\pi)$
5: $(y, q) \leftarrow \text{ASSIGNLABEL}(R^\star, b)$
Stratified Rejection Sampling
6: **if** $y = 1$ **and** $|E^+| < \kappa_{\text{pos}}$ **then** ▷ accept positive
7: $B \leftarrow B \cup \{b\};\ E^+ \leftarrow E^+ \cup \{q\}$
8: **else if** $y = 0$ **and** $|E^-| < \kappa_{\text{neg}}$ **then** ▷ accept negative
9: $B \leftarrow B \cup \{b\};\ E^- \leftarrow E^- \cup \{q\}$
10: **else**
11: **continue** ▷ reject sample
12: **end if**
13: **end while**
Synthesiser Output
14: task $\leftarrow (B, E^+, E^-)$
15: program $\leftarrow \text{VALIDATIONPROGRAM}(\text{task})$
16: prompt $\leftarrow \text{PROMPTGENERATOR}(\text{task})$
17: **return** ($R^\star$, program, prompt)

rule is a syntactically valid definite clause of the form $h\text{:-}b_1,\ \ldots,\ b_{R_{\text{len}}}$.

2. Background Synthesis (Alg. 1, lines 3-13). Once $R^\star$ is fixed, the synthesizer enters a loop to construct the background knowledge B and the label sets E^+ and E^-. This loop executes three steps until the desired number of positive and negative examples is reached:

(i) **Sample Background:** The BACKGROUNDGENERATOR samples a set of ground atoms specifying the properties and relationships of the background instance. The background sampling is directed by a probability distribution (B_π) over $\text{HB}_{\mathcal{G}}(\mathcal{V})$.

(ii) **Assign Label:** The ASSIGNLABEL function determines whether a query atom q (that is the ground atom of the target predicate h) is logically entailed by the sampled background b and the ground-truth rule $R^\star$ (i.e., whether $b \cup R^\star \models q$ holds). This produces a label (positive or negative) for the query. We denote the labeling function as:

$$\text{ASSIGNLABEL}(R^\star, b) = \begin{cases} (1, q), & \text{if } b \cup R^\star \models q \\ (0, q), & \text{otherwise} \end{cases}$$

(iii) **Accept/Reject Sample:** To ensure the desired class balance, a stratified rejection sampling strategy is used to populate the example sets. The generated background b and query q are accepted only if the corresponding example set (E^+ or E^-) is not yet full, as specified by the task size parameter (κ). If accepted, b is added to the task's main background knowledge B, and q is added to the appropriate example set. Otherwise, the sample is discarded.

Synthesizer Outputs (Alg. 1, lines 14-17). The synthesizer then finalizes three key outputs for each task: 1. The *latent ground-truth rule* ($R^\star$) of the task. 2. A *validation program*, i.e., an executable logic program encoding the task instance (B, E^+, E^-) for automated evaluation. 3. An *instruction prompt* that presents the task in a user-friendly format, either as natural language or Prolog code, ready for an LLM. App. Fig. 5 illustrates an example run to using the Synthesizer.

3.3 Training and Evaluation

The final stage, shown in Fig. 1 (right), uses the synthesized task to evaluate and train models.

SYMBOLICJUDGE. The core element for training and evaluation is the SymbolicJudge. It provides a deterministic interface for evaluating a candidate hypothesis H for a task. It runs H against the validation program, checking for correct entailment over all examples in E^+ and E^-. Two evaluation metrics are used: an overall binary score indicating whether the task is perfectly solved, and a partial score corresponding to the fraction of correctly classified examples.

$$\text{OVERALLSCORE}_{B,E^+,E^-}(H) = [\![\forall q \in E^+ : (B \cup H) \models q \ \wedge \ \forall q \in E^- : (B \cup H) \not\models q]\!] \in 0, 1$$

$$\text{PARTIALSCORE}_{B,E^+,E^-}(H) = \frac{\sum_{q \in E^+} [\![(B \cup H) \models q]\!] + \sum_{q \in E^-} [\![(B \cup H) \not\models q]\!]}{|E^+ \cup E^-|}$$

Model Evaluation. SLR also streamlines the creation of logical reasoning benchmark datasets for systematic model evaluation. By specifying various combinations of task language and configuration, users can generate diverse ILP tasks that span a broad range of domains, prompt styles, and reasoning complexities. Each synthesized task comprises a natural language prompt, the corresponding ground-truth rule, and an executable validation program for assessment by the Symbolic Judge.

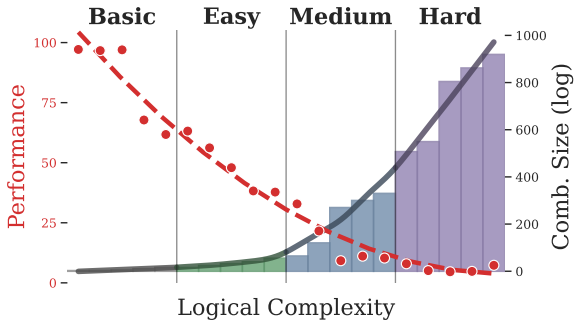


Figure 2: Overview of **SLR-BENCH**: The benchmark curriculum spans from basic to hard tasks with increasing logical and combinatorial complexity (bars, right y-axis). As logical complexity increases, Model performance (red, left y-axis) declines, highlighting current LLMs' limitations on more challenging reasoning tasks.

Model Training. SLR enables automated training loops, with model feedback available in two flavors. For SFT, the ground-truth rule $R^\star$ serves as the training target, allowing for cross-entropy loss updates based on predicted rules. For RLVR, the symbolic judge provides reward signals, using partial or overall scores, to guide the model's policy updates. This cohesive, automated pipeline efficiently supports the scalable generation, evaluation, and training of LLMs, facilitating systematic advances in logical reasoning capabilities.

Novelty and Systematic Generalization. SLR is fully synthetic, so overlap with existing data is statistically negligible, making it robust against data leakage and memorization. By default, we exclude ground-truth rules $R^\star$ in the test set that are found in the training set, guaranteeing that inputs and outputs at test time are completely novel to the model. This setup enables us to assess whether the model is capable of systematic generalization to entirely new rules.

4 SLR-BENCH: Instantiating SLR

SLR-BENCH is a concrete instantiation of the SLR framework that builds a 20-stage curriculum of logical reasoning problems (see Fig. 2). Each level specifies a language $\mathcal{L}$ and a set of configuration parameters θ, both of which are supplied directly to the system components described in Section 3, yielding 19k reasoning tasks with systematically increasing difficulty. Each stage contains 1k train, 10 eval, and 50 test samples, except for the first three levels, which have smaller training sets (26, 234, and 793) due to the limited task space; evaluation and test set sizes remain constant. An illustrative example for prompts and ground-truth rules can be found in Fig. 3.

Design rationale. The task specification is inspired by the train domain (Helff et al., 2025; Michalski, 1980; Mitchell, 1997), which is attractive for three reasons. First, its hierarchical object structure (trains $\rightarrow$ cars $\rightarrow$ attributes) naturally gives rise to first-order rules that are far richer than simple lookups, yet more tractable than general theorem proving. Second, every attribute has a small, discrete grounding domain, which allows us to measure and control the complexity of the problem precisely. Third, we ensure expandability and novelty, as SLR-BENCH is fully synthetic and programmatically generated from a user-constructed language and not curated from existing corpora.

Curriculum. SLR-BENCH presents a 20-level curriculum that progresses systematically in difficulty and is organized into four overarching complexity tiers: *basic*, *easy*, *medium*, and *hard*. For a detailed overview of the level-wise task specifications, see App. Sec. A and Tab. 3 Each level increases in difficulty through systematic modifications to both the task language and configuration, including: (i) increasing the overall size of the task (κ), (ii) expanding the vocabulary by adding new car constants per train, and (iii) broadening the set of predicates and grounding domains. Next, (iv) we adapt the policy used for synthesizing background knowledge, (v) alter the length R_{len} of the ground truth rule, and (vi) Adjust the rule sampling policy. Finally, (vii) the combinatorial space of tasks, quantified as the approximate number of distinct tasks that can be generated per level, also increases across the curriculum.

Language. The vocabulary in SLR-BENCH centers on two mutually exclusive class predicates, `eastbound/1` and `westbound/1`, which serve as the targets for classification tasks. Additionally, the vocabulary includes five predicates (`has_car/2`, `car_num/2`, `car_color/2`, `car_len/2`, and `has_wall/2`) that define the basic properties of trains, with their respective grounding domains fully specified in App. A.1. As curriculum levels increase in complexity, the vocabulary expands monotonically to accommodate new predicates and grounding domains at higher levels. These include additional categorical predicates (`has_roof/2`, `has_payload/2`,

Instruction & Task Prompt

```
You are a train classifier who is observing trains that are traveling either east- or westbound. Each train is
composed of one or more cars, and each car is characterized by a set of properties, represented as ground atoms over
a fixed set of predicates. The direction (eastbound or westbound) of a train is to be determined from its composition.
To describe the trains we define a set of predicates and grounding domains:

'has_car(Train, Car)': Specifies that 'Car' is part of the train 'Train'.
'car_num(Car, CarNumber)': Specifies the position of the car within its train. 'CarNumber' is a positive integer.
'car_color(Car, Color)': Specifies the color of the car. 'Color' can be 'red', 'blue', 'green', 'yellow', or 'white'.
'car_len(Car, Length)': Specifies the length of the car. 'Length' can be either 'short' or 'long'.
'has_wall(Car, WallType)': Specifies the wall type of a car. 'WallType' can be either 'full' or a 'railing'.

You are provided with positive and negative examples in the form of eastbound(t) or westbound(t) for each train t,
together with background knowledge consisting of ground facts over the above predicates which describe its composition.

eastbound(train0).                      westbound(train1).
has_car(train0, car0_1).                has_car(train1, car1_1).
car_num(car0_1, 1).                     car_num(car1_1, 1).
car_color(car0_1, red).                 car_color(car1_1, red).
car_len(car0_1, long).                  car_len(car1_1, short).
has_wall(car0_1, railing).              has_wall(car1_1, railing).

Your task is to formulate a hypothesis, i.e. a prolog rule of the form 'eastbound(Train) :- Body.' that correctly
distinguishes eastbound from westbound trains. The hypothesis must be true for all positive examples (i.e., eastbound
trains) and false for all negative examples (i.e., westbound trains). Aim to find the shortest correct rule, that is,
one that uses the fewest possible body literals subject to the prior constraints. Your rule must use only  predicates
defined above and must perfectly separate eastbound from westbound trains.
```

Ground-Truth Rule

```
eastbound(Train) :- has_car(Train,Car), car_len(Car,long)
```

Figure 3: Illustrative prompt and ground-truth rule generated by SLR (Level 1, SLR-BENCH). Language ($\mathcal{L}$): 5 predicates, 1 car per train. Task configuration (Θ): $\kappa = (1, 1)$ (one positive and one negative example); B_π = mirror; $R_{\text{len}} = 1$; R_{sample} = uniform. The prompt provides the full ILP instance, including background B, positive/negative examples (E^+, E^-), and natural-language instructions for the learning task.

`has_window/2`, `car_type/2`), and numerical predicates (`load_num/2`, `has_wheel/2`, `passenger_num/2`). Semantic coherence is maintained by constraining predicate groundings to valid combinations (e.g., only colors as arguments for `car_color`), and by enforcing mutually exclusive constraints across predicates (e.g., passenger cars cannot simultaneously carry payloads). Comprehensive details on the vocabulary and grammar can be found in App. A.1.

Task config. Next to the language $\mathcal{L}$, each curriculum level is parameterized using level-specific settings of θ that are summarized in App. Tab. 3 and fed directly to the SLR synthesizer (Alg. 1). Problem size (κ) increases steadily across levels, with an equal balance of positive and negative samples. For background knowledge, levels 1–5 use a *mirror* sampling policy to provide simple, nearly identical east- and westbound trains (differing only in ground atoms relevant to $R^\star$). From level 6 upward, background is sampled uniformly over the filtered Herbrand base, increasing train diversity. The first five (basic) levels employ uniform rule generation; from level 6 onwards, 30% of rules are LLM-guided, introducing greater diversity in variables, arithmetic, and recursion.

The compounding effect of larger problems, expanded vocabulary (constants and predicates), and more complex rules not only yields exponential growth in the combinatorial space, but also effectively increases the complexity of the tasks, while guaranteeing that heuristics that merely memorise surface patterns in early levels do not scale to later ones. For each generated task, we retain (i) the generated latent ground-truth rule, (ii) the corresponding validation program, and associated instruction prompt for the task. A representative example is shown in Fig. 3.

Intended use. SLR-BENCH serves two complementary purposes. (1) As a *static* benchmark, it allows fine-grained evaluation of an LLM's reasoning skills along an axis of increasing logical depth, and is easily extensible as models improve. (2) As a *dynamic* curriculum, it acts as the training backbone, providing reasoning tasks and model feedback for learning logical reasoning with LLMs and reasoning LLMs.

5 LLMs Can't Do Induction at Scale

We benchmark 17 LLMs on SLR-BENCH for their logical reasoning, syntactic capabilities, and computational efficiency across a systematic curriculum (*basic*, *easy*, *medium*, *hard*). In this section, we highlight the principal trends, identify the challenges revealed by our benchmark, and discuss the impact of training on logical reasoning data.

Table 2: SLR-BENCH leaderboard (DS version v0.1) with models evaluated across the 20-level curriculum. Each row reports a model's Logical Reasoning Level (LRL), syntax score, stage-specific logical reasoning accuracy (basic, easy, medium, hard), total completion tokens, and inference cost. Higher LRL and accuracy indicate superior logical reasoning, while lower compute values correspond to greater efficiency. Reasoning LLMs (orange) consistently outperform conventional LLMs (blue), while our logic-tuned LLMs (green) are closing the gap.

	LRL	Syntax	Logical-Reasoning Acc. (%)↑				Total Compute	
Model	(↑0-20)	Score (↑%)	Basic	Easy	Medium	Hard	Tokens (↓M)	Costs (↓$)
o3	**15.5**	80	**99**	93	**74**	**45**	4.30	207.24
o4-mini-high	12.8	88	98	**96**	40	21	4.62	24.24
o4-mini	12.3	86	93	88	52	13	3.98	21.43
o1	11.9	68	92	89	41	15	5.19	364.72
o3-mini	11.6	75	97	90	37	7	4.73	24.71
o4-mini-low	10.3	91	91	81	25	9	0.77	7.26
o1-mini	10.1	95	97	82	20	3	3.65	19.98
Gemini-thinking[1]	8.6	83	93	65	13	1	×	×
DeepSeek-R1[2]	8.1	57	89	61	9	3	1.95	1.15
gpt-4.5-prev	7.2	**100**	95	41	6	2	0.12	284.21
gpt-4o	6.7	96	90	37	6	1	0.16	10.49
Llama-3.3-70B	5.1	99	84	17	1	1	0.02	0.38
Llama-3.1-8B	5.0	87	82	17	1	1	**0.03**	0.08
Interlm2-20B	3.9	82	71	7	0	0	0.10	0.60
Qwen2-57B	3.9	81	71	7	0	0	0.08	2.72
CodeLlama-34b	3.5	78	68	1	0	0	0.09	3.53
Mixtral-8x7B	3.1	93	61	1	0	0	0.05	1.07
Llama-3.2-3B	1.6	61	31	1	0	0	0.27	**0.06**
Llama-3.1-8B FFT	(+4.4) 9.4	(+13) **100**	(+10) 92	(+60) 77	(+16) 17	(+1) 2	(+.01) 0.04	(+.01) 0.09
Llama-3.1-8B LoRA	(+3.4) 8.4	(+13) **100**	(+13) 95	(+40) 57	(+14) 15	(+0) 1	(+.01) 0.04	(+.01) 0.09
Llama-3.1-8B	5.0	87	82	17	1	1	**0.03**	0.08

×Data not available [1]Gemini-2.0-flash-thinking-exp-01-21 [2]DeepSeek-R1-Distill-Llama-70B

5.1 Evaluation Setup

All models are evaluated in a zero-shot fashion on SLR-BENCH using prompts generated in the previous section. Each model receives a single attempt per task. Due to the high computational costs for some of the models, we report results on a representative 600-task subset (which reduces our cost to just under $1,000). We report the following metrics: (i) **Logical–Reasoning Level (LRL):** The aggregated level of logical reasoning attained by a model on the 20-level SLR-BENCH:

$$\text{LRL} = \sum_{\ell=1}^{L} \frac{\#\text{solved}_\ell}{\#\text{tasks}_\ell}$$

where L is the number of curriculum levels. (ii) **Syntax Score:** The proportion of syntactically valid logic rules generated by the model. (iii) **Logic-Reasoning Accuracy:** Fraction of correct solutions per complexity tier and overall. (iv) **Compute:** Aggregate completion tokens and cost in USD (see App. Tab. 4 for pricing details).

5.2 Training Setup

We fine-tune models on 16,000 training tasks sampled from all levels of the SLR-BENCH curriculum. For each task, the input consists of the complete prompt, while the output target is the ground-truth rule $R^\star$. All training instances are strictly disjoint from the test set to prevent data leakage and ensure fair evaluation. Thus, neither the evaluation prompts nor the ground-truth rules have been seen during training. We explore two training regimes for Llama-3-8B-Instruct: (1) full-parameter fine-tuning (FFT) and (2) parameter-efficient adaptation via LoRA tuned on a shuffled version of the dataset. Both approaches optimize a cross-entropy loss between the model's predicted rule tokens and the ground truth.

5.3 Analysis and Key Findings

Tab. 2 reports the overall model performance across all 20 curriculum levels, with the compute-performance trade-off for OpenAI models in Fig. 4.

Curriculum Stages Effectively Modulate Challenge. SLR-BENCH's staged curriculum successfully delivers a controlled gradient in task complexity. At the *basic* tier, most models achieve accuracies close to 100% (see Tab. 2), confirming that basic logical inference is a solved problem for contemporary LLMs. As the curriculum progresses, each new tier introduces greater logical depth, leading to a monotonic decrease in model performance on the subsequent complexity tiers.

At the *easy* tier, a clear gap emerges between the conventional LLMs as well as the reasoning LLMs. While conventional LLMs plateau (all models solve less than 50%), reasoning models sustain performance, solving most of the easy tasks (with all models above 60% solved). This indicates that the conventional LLMs have much stronger difficulties when it comes to solving logical reasoning tasks, and are insufficient for more advanced logical induction.

At the *medium* tier, difficulty increases further, causing significant performance drops even among reasoning models. Only 2 models (o3 and 04-mini) are still able to clear more than 50% of the tasks. Among OpenAI's mini series, Tab. 2 depicts a clear trend: steady improvement across model generations. While o1-mini solves only 20% of medium tasks, o3-mini 37%, and o4-mini reaches 52%. This illustrates both progress across model generations and the fine granularity and challenge provided by the benchmark's curriculum.

The *hard* tier proves formidable for all current models. Even the strongest model (OpenAI's o3) solves fewer than half of these tasks (45%), and all others fall below 21%, demonstrating that the upper curriculum levels set a bar that remains out of reach for today's LLMs.

Reasoning is Challenging; Syntax is Nearly Solved. Syntactic validity is nearly perfect for most LLMs. Especially base LLMs typically produce syntactically valid logic rules (see Tab. 2). In contrast, the logical reasoning accuracy of all base LLMs lags by 50+ percentage points. Most errors, therefore, are of a semantic nature. Interestingly, advanced reasoning models (DeepSeek-R1, o3) exhibit lower syntax scores (57%, 80%) than base models (e.g., Llama-3-70B, GPT-4.5-prev at 99% and 100%). This degradation often stems from models producing ill-formed rules when navigating complex reasoning traces. Nevertheless, across all models, semantic errors are the predominant obstacles to higher accuracy.

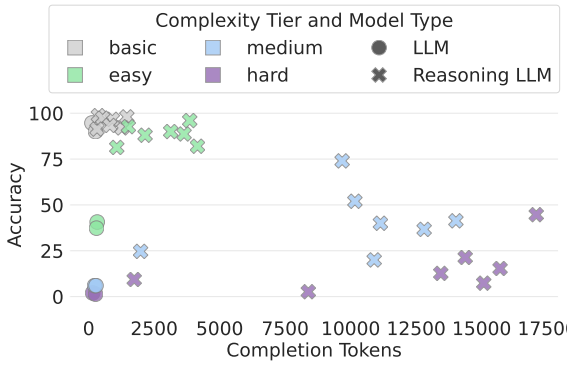


Figure 4: Compute–performance trade-off for OpenAI models. Reasoning LLMs typically achieve higher accuracy than base LLMs, but require greater test-time compute. More completion tokens do not always lead to higher accuracy, though more complex tasks generally demand more completion tokens.

Scaling Test-time Compute Improves Reasoning, but Returns Diminish and Costs Escalate. Increasing test-time compute (via reasoning models) yields substantial gains in logical reasoning performance. For example, the lowest-performing reasoning LLM (DeepSeek-R1, LRL=8.1) surpasses the best conventional LLM (GPT-4.5, LRL=7.2). However, improvements come at a significant computational expense. For instance, moving from GPT-4o to o3 doubles accuracy, but inflates cost by 20 times and increases the number of generated completion tokens by 27 times. Analyzing the number of generated reasoning tokens (Fig. 4), we find that higher test-time compute generally correlates with better reasoning outcomes, but this relationship does not always hold. On the easy and medium tiers, models that generate the most completion tokens do not achieve the highest reasoning accuracy. This trend persists even within the same architecture (*cf.* Tab. 2). For instance, o4-mini-high typically outperforms o4-mini overall (LRL: 12.3 vs. 12.8), yet on the medium tier, o4-mini-high lags behind (40% vs. 52% accuracy). This plateau effect demonstrates that, beyond a certain threshold, additional compute may yield diminishing, or even negative, returns in reasoning accuracy.

Scaling Model Parameters Translates to only Marginal Gains in Logical Reasoning. Scaling model parameters across both open and closed checkpoints consistently results in diminishing returns when it comes to logical reasoning. For instance, Llama-3-70B surpasses its 8B counter-

part by just 1%, while GPT-4.5-prev shows only a 2% advantage over GPT-4o. These marginal gains demonstrate that simply scaling model size does not guarantee corresponding advances in logical proficiency.

Computational Burden Increases with Task Complexity. As we move towards more complex reasoning problems, we observe that reasoning models increase their spend tokens (Fig. 4). The increased token usage translates into higher test-time computational and financial costs. Consequently, scaling LLM-based reasoning to harder tasks (via test-time compute) is fundamentally limited by these growing resource demands.

Logic-tuning with SLR Improves Reasoning. Crucially, to test for systematic generalization to entirely novel tasks (and avoid memorization), we test our models on held-out data, ensuring that both input prompts and ground-truth rules are never seen during training. Via logic-tuning, our models not only outperform all base LLMs but also surpass some advanced reasoning models such as Gemini-2.0-flash-thinking, demonstrating that base language models can indeed acquire logical reasoning skills. Substantial performance gains are especially evident at lower difficulty tiers, as the accuracy on the "easy" tier increases from just 17% to 77%. However, improvements at the hardest tier are limited (e.g., accuracy rises only from 1% to 2%), suggesting that these challenges may exceed the learning capacity of the current model architecture. Overcoming this limitation may require architectural changes or the adoption of more sophisticated, search-based reasoning strategies. Overall, our approach yields significant performance improvements while using only a fraction of the inference tokens, e.g., `o1-mini` achieves only a marginally higher LRL (10.1 vs. 9.4,), but does so at the cost of 91 times more tokens (*cf.* Tab. 2).

6 Conclusion and Future Direction

In this work, we introduced SLR, a principled and fully automated framework for synthesizing, training on, and evaluating logical reasoning benchmarks for LLMs. By leveraging symbolic synthesis and evaluation, our approach enables precise control over problem complexity and guarantees dataset novelty, circumvents issues of data leakage, and ambiguous grading. Using SLR, we developed SLR-BENCH: a 20-level curriculum spanning over 19k tasks with progressively increasing logical complexity.

Our evaluation reveals that while current LLMs readily produce syntactically valid logic rules, robust logical reasoning remains elusive for conventional models, particularly as task complexity scales. Recent reasoning LLMs close some of the gap, but do so at large computational costs. In contrast, we find that logic-tuning on SLR-BENCH yields substantial improvements, achieving strong performance at a fraction of the inference costs. Furthermore, we hypothesize that combining the SLR-BENCH curriculum with reinforcement learning using verifiable rewards (RLVR) could further enhance reasoning performance. We are actively investigating the integration of SLR-BENCH into established training pipelines for both standard and reasoning LLMs, with a comprehensive evaluation of their impact reserved for future work.

Looking forward, SLR paves the way for several promising research directions: expanding into richer logical domains beyond the train scenario, supporting open-ended program synthesis and multi-rule induction, benchmarking neuro-symbolic and interactive reasoning systems, and moving toward higher-order tasks such as causal inference and theory of mind. Ultimately, SLR provides a flexible and extensible resource for probing and advancing the frontiers of logical reasoning in the next generation of language models.

7 Limitations

While SLR and SLR-BENCH provide a scalable testbed for logical reasoning, there remain many opportunities for further enrichment. Although SLR-BENCH currently applies SLR to the train domain with a single rule, the framework is readily extensible to multiple more complex, multi-rule reasoning scenarios and to entirely different domains. Our current focus on first-order, function-free Horn clauses enables systematic benchmark creation and evaluation; future instantiations could expand towards higher-order logic or probabilistic reasoning. While synthetic task generation comes with many benefits, such as ensuring novelty and precise control, it makes it difficult to incorporate real-world diversity and ambiguity. Our symbolic judge provides deterministic, discrete scoring and could potentially be enhanced to also recognize partial solutions, syntactically invalid rules, or natural language formulations. Overall, these points high-

light the flexibility of our framework and outline promising directions for broadening its reach and impact.

Broader Impact

SLR and SLR-BENCH provide a scalable, reproducible foundation for evaluating and advancing logical reasoning in AI without relying on human annotation. By enabling robust measurement and targeted training, our framework supports progress in areas such as scientific discovery, program synthesis, and trustworthy AI. However, as LLMs acquire deeper logical competence, the risk of dual-use knowledge increases, enabling beneficial applications but also the potential for misuse, such as generating deceptive arguments or bypassing safety mechanisms. We urge responsible use and active consideration of ethical risks as these capabilities advance.

References

Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. 2023. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, page 41–48, New York, NY, USA. Association for Computing Machinery.

Victor Bursztyn, David Demeter, Doug Downey, and Larry Birnbaum. 2022. Learning to perform complex tasks through compositional fine-tuning of language models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 1676–1686, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *ArXiv*, abs/1803.05457.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, and 181 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *Preprint*, arXiv:2501.12948.

Quentin Delfosse, Jannis Blüml, Fabian Tatai, Théo Vincent, Bjarne Gregori, Elisabeth Dillies, Jan Peters, Constantin Rothkopf, and Kristian Kersting. 2025. Deep reinforcement learning agents are not even close to human intelligence. *Preprint*, arXiv:2505.21731.

Siqi Fan, Peng Han, Shuo Shang, Yequan Wang, and Aixin Sun. 2025. Cothink: Token-efficient reasoning via instruct models guiding reasoning models. *Preprint*, arXiv:2505.22017.

Clémentine Fourrier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. 2024. Open llm leaderboard v2. `https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard`.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, and 5 others. 2024. The language model evaluation harness.

Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Luke Benson, Lucy Sun, Ekaterina Zubova, Yujie Qiao, Matthew Burtell, David Peng, Jonathan Fan, Yixin Liu, Brian Wong, Malcolm Sailor, Ansong Ni, Linyong Nan, Jungo Kasai, Tao Yu, and 7 others. 2022. Folio: Natural language reasoning with first-order logic. *arXiv preprint arXiv:2209.00840*.

Lukas Helff, Wolfgang Stammer, Hikaru Shindo, Devendra Singh Dhami, and Kristian Kersting. 2025. V-lol: A diagnostic dataset for visual logical learning. *Journal of Data-centric Machine Learning Research*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021a. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021b. Measuring mathematical problem solving with the math dataset. *NeurIPS*.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, Lalit K. Jain, Virginia Aglietti, Disha Jindal, Peter Chen, Nishanth Dikkala, Gladys Tyen, Xin Liu, Uri Shalit, Silvia Chiappa, Kate Olszewska, Yi Tay, Vinh Q. Tran, Quoc V. Le, and Orhan Firat. 2025. Big-bench extra hard. *Preprint*, arXiv:2502.19187.

Jiin Kim, Byeongjun Shin, Jinha Chung, and Minsoo Rhu. 2025. The cost of dynamic reasoning: Demystifying ai agents and test-time scaling from an ai infrastructure perspective. *Preprint*, arXiv:2506.04301.

Aida Kostikova, Zhipin Wang, Deidamea Bajri, Ole Putz, Benjamin Paassen, and Steffen Eger. 2025. Lllms: A data-driven survey of evolving research on limitations of large language models.

Bill Yuchen Lin. 2024. Zeroeval: A unified framework for evaluating language models.

Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. 2025. Zebralogic: On the scaling limits of llms for logical reasoning.

Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Faeze Brahman, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. 2024. Wildbench: Benchmarking llms with challenging tasks from real users in the wild. *Preprint*, arXiv:2406.04770.

Hanmeng Liu, Jian Liu, Leyang Cui, Zhiyang Teng, Nan Duan, Ming Zhou, and Yue Zhang. 2023. Logiqa 2.0—an improved dataset for logical reasoning in natural language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:2947–2962.

Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. 2020. Logiqa: A challenge dataset for machine reading comprehension with logical reasoning. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 3622–3628. International Joint Conferences on Artificial Intelligence Organization.

Junteng Liu, Yuanxiang Fan, Zhuo Jiang, Han Ding, Yongyi Hu, Chi Zhang, Yiqi Shi, Shitong Weng, Aili Chen, Shiqi Chen, Yunan Huang, Mozhi Zhang, Pengyu Zhao, Junjie Yan, and Junxian He. 2025. Synlogic: Synthesizing verifiable reasoning data at scale for learning logical reasoning and beyond. *Preprint*, arXiv:2505.19641.

Man Luo, Shrinidhi Kumbhar, Ming shen, Mihir Parmar, Neeraj Varshney, Pratyay Banerjee, Somak Aditya, and Chitta Baral. 2024. Towards logiglue: A brief survey and a benchmark for analyzing logical reasoning capabilities of language models. *Preprint*, arXiv:2310.00836.

Kaijing Ma, Xinrun Du, Yunran Wang, Haoran Zhang, Zhoufutu Wen, Xingwei Qu, Jian Yang, Jiaheng Liu, Minghao Liu, Xiang Yue, Wenhao Huang, and Ge Zhang. 2024. Kor-bench: Benchmarking language models on knowledge-orthogonal reasoning tasks. *Preprint*, arXiv:2410.06526.

Meta, Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, and 543 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Ryszard S. Michalski. 1980. Pattern recognition as rule-guided inductive inference. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-2(4):349–361.

Tom Michael Mitchell. 1997. Machine learning, international edition. In *McGraw-Hill Series in Computer Science*.

Philipp Mondorf and Barbara Plank. 2024. Liar, liar, logical mire: A benchmark for suppositional reasoning in large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7114–7137, Miami, Florida, USA. Association for Computational Linguistics.

Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2024. Enhancing reasoning capabilities of llms via principled synthetic logic corpus. In *Annual Conference on Neural Information Processing Systems*.

OpenAI. 2025. Openai o3 and o4-mini system card. https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, and 262 others. 2024. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

Tafjord Oyvind, Dalvi Bhavana, and Clark Peter. 2021. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634. Association for Computational Linguistics.

Bhrij Patel, Souradip Chakraborty, Wesley A. Suttle, Mengdi Wang, Amrit Singh Bedi, and Dinesh Manocha. 2024a. Aime: Ai system optimization via multiple llm evaluators. *Preprint*, arXiv:2410.03131.

Nisarg Patel, Mohith Kulkarni, Mihir Parmar, Aashna Budhiraja, Mutsumi Nakamura, Neeraj Varshney, and Chitta Baral. 2024b. Multi-logieval: Towards evaluating multi-step logical reasoning ability of large language models. *Preprint*, arXiv:2406.17169.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, Michael Choi, Anish Agrawal, Arnav Chopra, Adam Khoja, Ryan Kim, Richard Ren, Jason Hausenloy, Oliver Zhang, Mantas Mazeika, and 1090 others. 2025. Humanity's last exam. *Preprint*, arXiv:2501.14249.

Shanghaoran Quan, Jiaxi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, and 1 others. 2025.

Codeelo: Benchmarking competition-level code generation of llms with human-comparable elo ratings. *arXiv preprint arXiv:2501.01257*.

Luc De Raedt. 1997. Logical settings for concept-learning. *Artif. Intell.*, 95:187–201.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2024. GPQA: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling*.

Abulhair Saparov and He He. 2023. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *The Eleventh International Conference on Learning Representations*.

Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. 2025. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity.

Koustuv Sinha, Shagun Sodhani, Jin Dong, Joelle Pineau, and William L. Hamilton. 2019. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 4506–4515, Hong Kong, China. Association for Computational Linguistics.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2023. Challenging big-bench tasks and whether chain-of-thought can solve them. In *ACL (Findings)*, pages 13003–13051.

Jason Weston, Antoine Bordes, Sumit Chopra, and Tomas Mikolov. 2015. Towards ai-complete question answering: A set of prerequisite toy tasks. *arXiv: Artificial Intelligence*.

Tim Woydt, Moritz Willig, Antonia Wust, Lukas Helff, Wolfgang Stammer, Constantin A. Rothkopf, and Kristian Kersting. 2025. Fodor and pylyshyn's legacy – still no human-like systematic compositionality in neural networks.

Wüst, Woydt, Helff, Dhami, Rothkopf, and Kersting. 2025. Bongard in wonderland: Visual puzzles that still make ai go mad? In *Proceedings of the 42st International Conference on Machine Learning (ICML)*.

Chulin Xie, Yangsibo Huang, Chiyuan Zhang, Da Yu, Xinyun Chen, Bill Yuchen Lin, Bo Li, Badih Ghazi, and Ravi Kumar. 2024. On memorization of large language models in logical reasoning. *Preprint*, arXiv:2410.23123.

Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. 2025. Logic-rl: Unleashing llm reasoning with rule-based reinforcement learning. *Preprint*, arXiv:2502.14768.

Nathan Young, Qiming Bao, Joshua Bensemann, and Michael Witbrock. 2022. AbductionRules: Training transformers to explain unexpected inputs. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 218–227, Dublin, Ireland. Association for Computational Linguistics.

Yujun Zhou, Jiayi Ye, Zipeng Ling, Yufei Han, Yue Huang, Haomin Zhuang, Zhenwen Liang, Kehan Guo, Taicheng Guo, Xiangqi Wang, and Xiangliang Zhang. 2025. Dissecting logical reasoning in llms: A fine-grained evaluation and supervision study. *Preprint*, arXiv:2506.04810.

A Task Specification

Each task in SLR-BENCH is precisely governed by a combination of language features and task configuration parameters, enabling fine-grained control over complexity and diversity. A task specification comprises two main components: (i) the logical language, which determines the set of predicates and argument types available, and (ii) the task configuration, which defines structural aspects of the task such as problem size, background knowledge sampling, rule length, sampling strategy, and the combinatorial space of realizable tasks. Tab. 3 details the curriculum's level-wise specifications, showing how both language elements and the task config to create the individual levels.

A.1 Language

Predicates and Types. SLR defines a flexible, extensible vocabulary to support the systematic generation and evaluation of logical reasoning tasks. The primary predicate signatures and their argument types used in SLR-BENCH are:

$$\texttt{eastbound}(\mathit{TRAIN})$$
$$\texttt{westbound}(\mathit{TRAIN})$$
$$\texttt{has_car}(\mathit{TRAIN}, \mathit{CAR})$$
$$\texttt{car_num}(\mathit{CAR}, \mathit{NUM})$$
$$\texttt{car_color}(\mathit{CAR}, \mathit{COLOR})$$
$$\texttt{car_len}(\mathit{CAR}, \mathit{LEN})$$
$$\texttt{has_wall}(\mathit{CAR}, \mathit{WALL})$$
$$\texttt{has_roof}(\mathit{CAR}, \mathit{ROOF})$$
$$\texttt{has_payload}(\mathit{CAR}, \mathit{LOADS})$$
$$\texttt{load_num}(\mathit{CAR}, \mathit{NPAY})$$
$$\texttt{has_wheel}(\mathit{CAR}, \mathit{WHEELS})$$
$$\texttt{has_window}(\mathit{CAR}, \mathit{WINDOW})$$
$$\texttt{car_type}(\mathit{CAR}, \mathit{CTYPE})$$

Grounding Domains. Each argument type is grounded in a finite set of discrete constants:

$$\begin{aligned}
\mathit{NUM} &::= [\![\texttt{0-9}]\!]+ \\
\mathit{CAR} &::= [\![\texttt{0-9}]\!]+ \\
\mathit{COLOR} &::= \texttt{red} \mid \texttt{blue} \mid \texttt{green} \mid \\
&\quad \texttt{yellow} \mid \texttt{white} \\
\mathit{LEN} &::= \texttt{short} \mid \texttt{long} \\
\mathit{WALL} &::= \texttt{full} \mid \texttt{railing} \\
\mathit{ROOF} &::= \texttt{roof_foundation} \mid \texttt{solid_roof} \mid \\
&\quad \texttt{braced_roof} \mid \texttt{peaked_roof} \mid \texttt{none} \\
\mathit{WHEELS} &::= \texttt{2} \mid \texttt{3} \\
\mathit{LOADS} &::= \texttt{blue_box} \mid \texttt{golden_vase} \mid \\
&\quad \texttt{barrel} \mid \texttt{diamond} \mid \texttt{metal_pot} \mid \\
&\quad \texttt{oval_vase} \mid \texttt{none} \\
\mathit{NPAY} &::= \texttt{0} \mid \texttt{1} \mid \texttt{2} \mid \texttt{3} \\
\mathit{WINDOW} &::= \texttt{full} \mid \texttt{half} \mid \texttt{none} \\
\mathit{CTYPE} &::= \texttt{passenger} \mid \texttt{freight} \mid \texttt{mixed} \\
\mathit{NPAX} &::= [\![\texttt{0-9}]\!]
\end{aligned}$$

Grammar Constraints. Predicates are only instantiated with semantically compatible constant types. For example, $\texttt{car_color}(\cdot, \cdot)$ only takes car objects and color constants as arguments; ill-typed facts are excluded during synthesis.

B Compute Costs and Model Pricing

Compute Costs. Compute costs are reported as the total USD cost to run all prompts, based on publicly listed API prices as of 01.05.2025. Pricing ignores server-side token caching, as actual cache hit counts are unavailable. Table 4 summarizes per-model cost rates and API sources.

C Example: Task Synthesis Process

Table 3: SLR-BENCH Curriculum: level-wise configurations, detailing language and task parameters for each difficulty stage. Language complexity is systematically increased by expanding the number of car constants and predicates. Task configuration grows via adapting problem size, background sampling, rule length, and rule sampling strategy. The final column reports the approximate combinatorial size of unique tasks available at each level.

	Level	Language		Task Configuration				Comb. size
		$\#Const.$	$\#Pred.$	κ	B_π	R_{len}	R_{sample}	
Basic	1	1	5	2	mirror	1	uniform	10^3
	2	1	5	2	mirror	1-2	uniform	10^3
	3	1	5	4	mirror	1-2	uniform	10^5
	4	2	5	4	mirror	1-2	uniform	10^{10}
	5	2	5	6	mirror	1-2	uniform	10^{16}
Easy	6	2	5	6	uniform	1-2	uniform/llm	10^{16}
	7	2	6	6	uniform	1-2	uniform/llm	10^{24}
	8	2-3	6	8	uniform	1-2	uniform/llm	10^{32}
	9	2-3	6	10	uniform	2-3	uniform/llm	10^{40}
	10	2-3	7	12	uniform	2-3	uniform/llm	10^{55}
Medium	11	2-4	7	14	uniform	2-3	uniform/llm	10^{65}
	12	2-4	9	16	uniform	3-4	uniform/llm	10^{120}
	13	4-6	9	18	uniform	3-4	uniform/llm	10^{271}
	14	4-6	9	20	uniform	4-5	uniform/llm	10^{300}
	15	4-6	9	22	uniform	4-5	uniform/llm	10^{330}
Hard	16	5-6	10	24	uniform	4-5	uniform/llm	10^{507}
	17	5-6	10	26	uniform	4-5	uniform/llm	10^{549}
	18	5-6	12	28	uniform	4-5	uniform/llm	10^{805}
	19	5-6	12	30	uniform	5	uniform/llm	10^{861}
	20	5-6	12	32	uniform	5	uniform/llm	10^{919}

Table 4: Model Pricing ($ per 1M tokens). API rates as of 01.05.2025.

Model	Model Tag	Input	Input(Cached)	Output	API
gpt-4.1	gpt-4.1-2025-04-14	2.00	0.50	8.00	OpenAI
gpt-4.1-mini	gpt-4.1-mini-2025-04-14	0.40	0.10	1.60	OpenAI
gpt-4.1-nano	gpt-4.1-nano-2025-04-14	0.10	0.025	0.40	OpenAI
gpt-4.5-preview	gpt-4.5-preview-2025-02-27	75.00	37.50	150.00	OpenAI
gpt-4o	gpt-4o-2024-08-06	2.50	1.25	10.00	OpenAI
gpt-4o-mini	gpt-4o-mini-2024-07-18	0.15	0.075	0.60	OpenAI
o1	o1-2024-12-17	15.00	7.50	60.00	OpenAI
o1-pro	o1-pro-2025-03-19	150.00		600.00	OpenAI
o3	o3-2025-04-16	10.00	2.50	40.00	OpenAI
o4-mini-low	o4-mini-2025-04-16	1.10	0.275	4.40	OpenAI
o4-mini	o4-mini-2025-04-16	1.10	0.275	4.40	OpenAI
o4-mini-high	o4-mini-2025-04-16	1.10	0.275	4.40	OpenAI
o3-mini	o3-mini-2025-01-31	1.10	0.55	4.40	OpenAI
o1-mini	o1-mini-2024-09-12	1.10	0.55	4.40	OpenAI
DeepSeek-R1	DeepSeek-R1-Distill-Llama-70B	0.10		0.40	OpenRouter
Internlm2-20b	Internlm2-20b	0.15		0.20	Estimated
Llama-3.2-3B	Llama-3.2-3B-Instruct	0.015		0.025	OpenRouter
Llama-3.1-8B	Llama-3.1-8B-Instruct	0.02		0.03	OpenRouter
Llama-3.3-70B	Llama-3.3-70B-Instruct	0.10		0.25	OpenRouter
Mixtral-8x7B	Mixtral-8x7B-Instruct-v0.1	0.24		0.24	OpenRouter
Qwen2-57B-A14B	Qwen2-57B-A14B-Instruct	0.70		0.70	Estimated
QwQ-32B	QwQ-32B-Preview	0.15		0.20	OpenRouter
CodeLlama-34b	CodeLlama-34b-Instruct-hf	0.776		0.776	TogetherAI
MetaTuned Llama	Llama-3.1-8B-Tuned-FFT	0.02		0.03	OpenRouter
MetaTuned Llama LoRA	Llama-3.1-8B-Tuned-LoRA	0.02		0.03	OpenRouter

Synthesis Process and Outputs

Task Specification:

(i) **Language** $\mathcal{L} = (\mathcal{V}, \mathcal{G})$:

- Vocabulary $\mathcal{V}$: Predicates $\mathcal{P}$ = {is_red_train/1, has_car/2, car_color/2, car_len/2}; Constants $\mathcal{C}$ = {t1, t2, c1, c2, red, blue, short, long}
- Grammar $\mathcal{G}$: Restricts predicates to apply to compatible constant types.

(ii) **Configuration** Θ: Rule length $R_{\text{len}} = 2$; Problem size $\kappa = (\kappa_{\text{pos}} = 1,\ \kappa_{\text{neg}} = 1)$

Synthesis Steps:

1. Rule Synthesis: The RULEGENERATOR produces a latent ground-truth rule $R^\star$:

```
is_red_train(T) :- has_car(T, C), car_color(C, red).
```

2. Background Synthesis (Loop):

Iteration 1 (finds a positive example):

- Sample Background (b_1): 'has_car(t1, c1). car_color(c1, red).'
- Assign Label: Query q_1 = *is_red_train(t1)*. Entailment $b_1 \cup R^\star \models q_1$ holds. Result: $(1, q_1)$.
- Accept/Reject: $|E^+| < \kappa_{\text{pos}}$, sample is **accepted**. $B \leftarrow b_1$, $E^+ \leftarrow \{q_1\}$.

Iteration 2 (finds a negative example):

- Sample Background (b_2): 'has_car(t2, c2). car_color(c2, blue).'
- Assign Label: Query q_2 = *is_red_train(t2)*. Entailment $b_2 \cup R^\star \models q_2$ fails. Result: $(0, q_2)$.
- Accept/Reject: $|E^-| < \kappa_{\text{neg}}$, sample is **accepted**. $B \leftarrow B \cup b_2$, $E^- \leftarrow \{q_2\}$.

The loop terminates as both target sizes are met. The final task is $\mathcal{I} = (B, E^+, E^-)$.

Final Synthesizer Outputs:

1. Latent Ground-Truth Rule ($R^\star$):

```
is_red_train(T) :- has_car(T, C), car_color(C, red).
```

2. Validation Program (B, E^+, E^-):

```
has_car(t1, c1).
car_color(c1, red).
has_car(t2, c2).
car_color(c2, blue).
is_red_train(t1).
```

3. Instruction Prompt (example formats):

(a) Prolog-style Prompt:

```
% Given the following background knowledge:
has_car(t1, c1).
car_color(c1, red).
has_car(t2, c2).
car_color(c2, blue).
is_red_train(t1).
% Your task is to find a rule "is_red_train(T) :-" that solves the bk.
```

(b) Natural Language Prompt:

```
% Given the following background knowledge:
Train t1 has a car c1. The car c1 is red.
Train t2 has a car c2. The car c2 is blue.
% Your task is to find a rule "is_red_train(T) :-" that solves the bk.
```

Figure 5: Step-by-step example of the automatic ILP task synthesis process in SLR. Given a task specification, comprising a language and a task config, the synthesizer generates a ground-truth rule, samples background knowledge, assigns positive and negative example labels, and produces symbolic (Prolog-style) or natural-language prompts. The figure illustrates all intermediate steps and the final output of the synthesizer.